Project Report: Real-Time Chat Application with WebSockets

1. Executive Summary

This project is a high-performance, real-time messaging platform developed as part of the technical assignment for Smart Winner. Utilizing the MERN Stack (MongoDB, Express, React, Node.js) and Socket.io, the application provides a seamless communication experience with features like private messaging, group creation, and multi-format media sharing.

2. Technical Stack

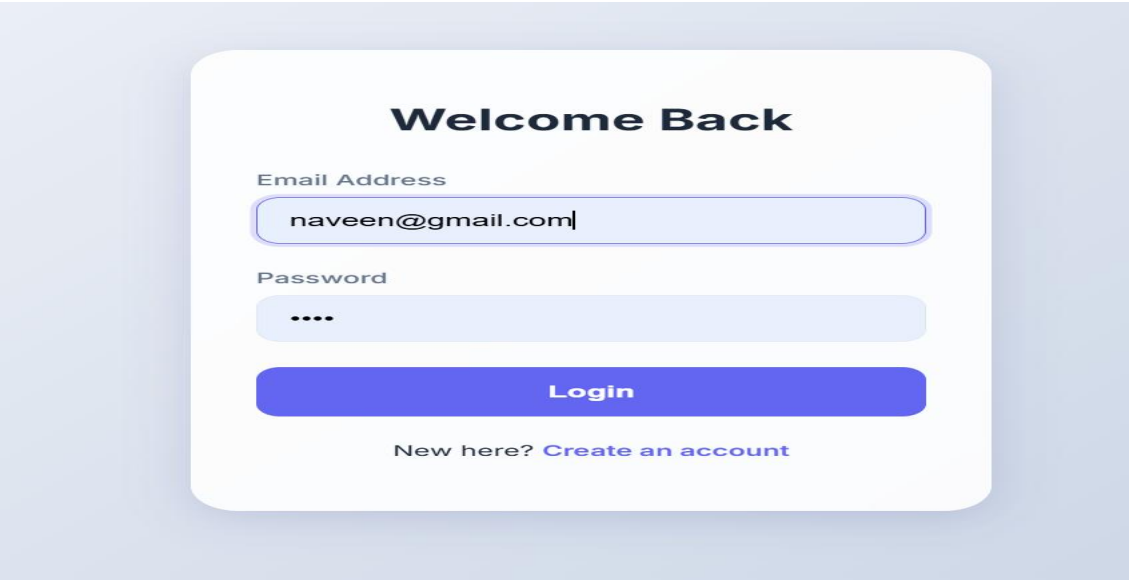
* Frontend: React.js (v19), Vite (v8)
* Backend: Node.js, Express.js (v5)
* Real-time Engine: Socket.io (v4) for WebSockets
* Database: MongoDB (v9) with Mongoose ODM
* Security: JWT (JSON Web Tokens) & Bcrypt password hashing
* File Handling: Multer middleware for media uploads

3. Feature Walkthrough & Visuals

3.1 User Authentication

The application features a secure entry point. Users can register with a unique username and email. Data is protected

using JWT-based authentication to ensure that chat history remains private.



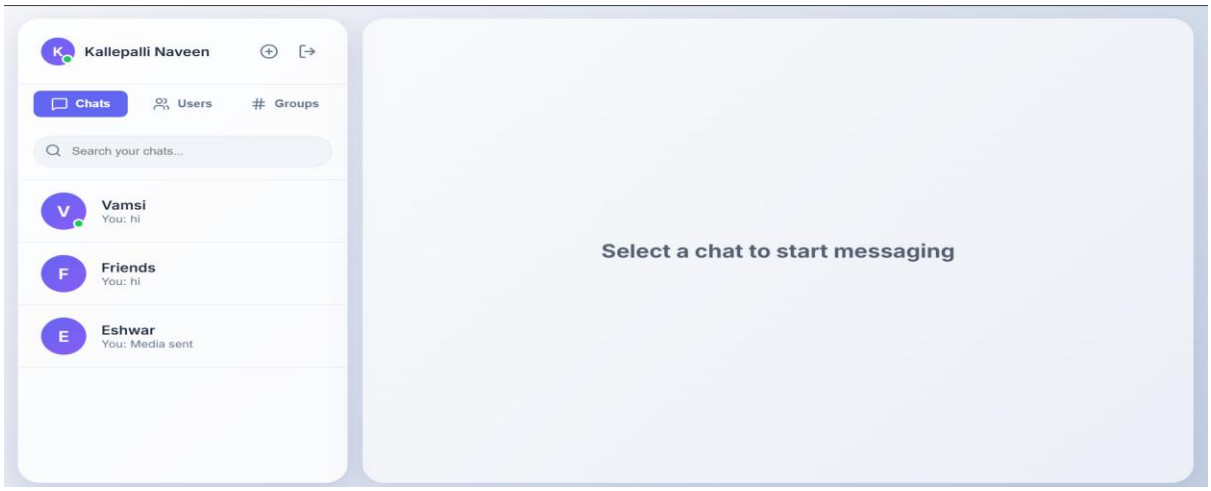
3.2 Main Dashboard & Navigation

The sidebar provides a unified view of all conversations. It includes three dedicated tabs:

* Chats: Current active conversations with "Last Message" previews.

* Users: Global user search to start new private chats.

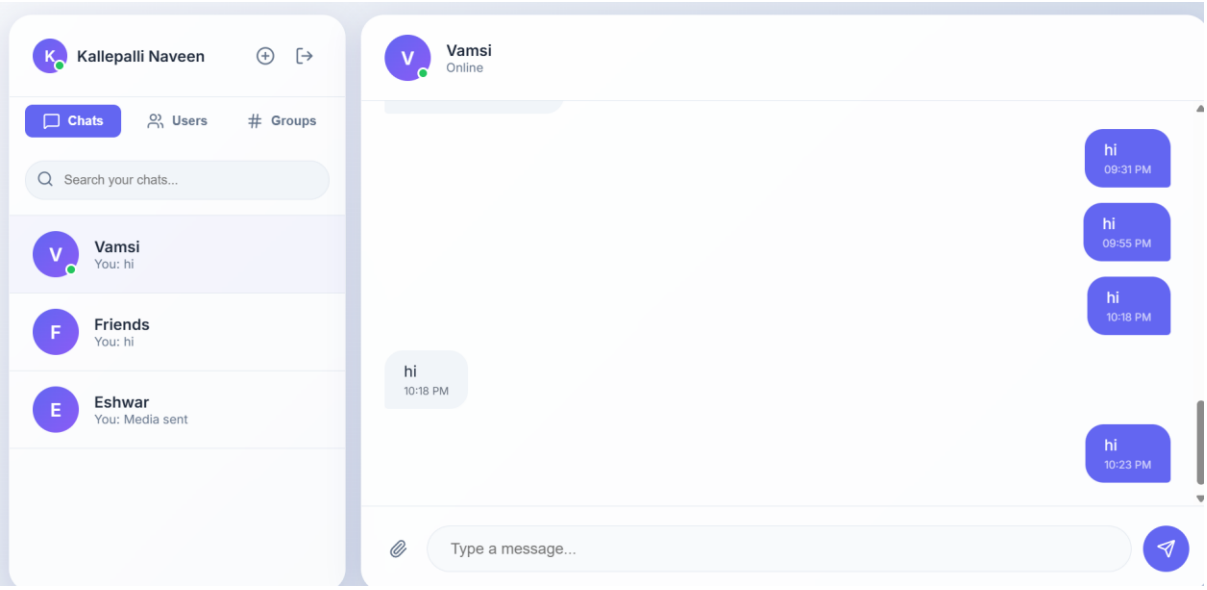
* Groups: Unified search for joined groups and global discovery.



3.3 Real-Time Private Messaging

Instant delivery is achieved through WebSockets. The system includes Online/Offline indicators so users can see who is

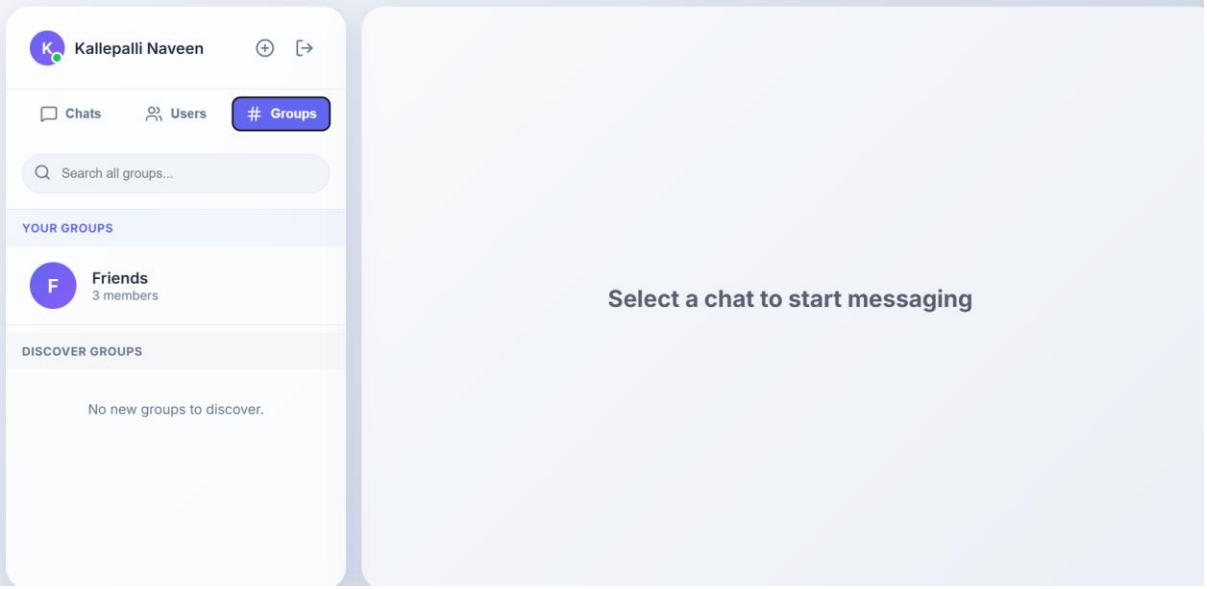
currently active on the platform.



3.4 Group Management & Discovery

The "Groups" tab allows users to find groups they are already members of while also discovering public groups they can

join with a single click.

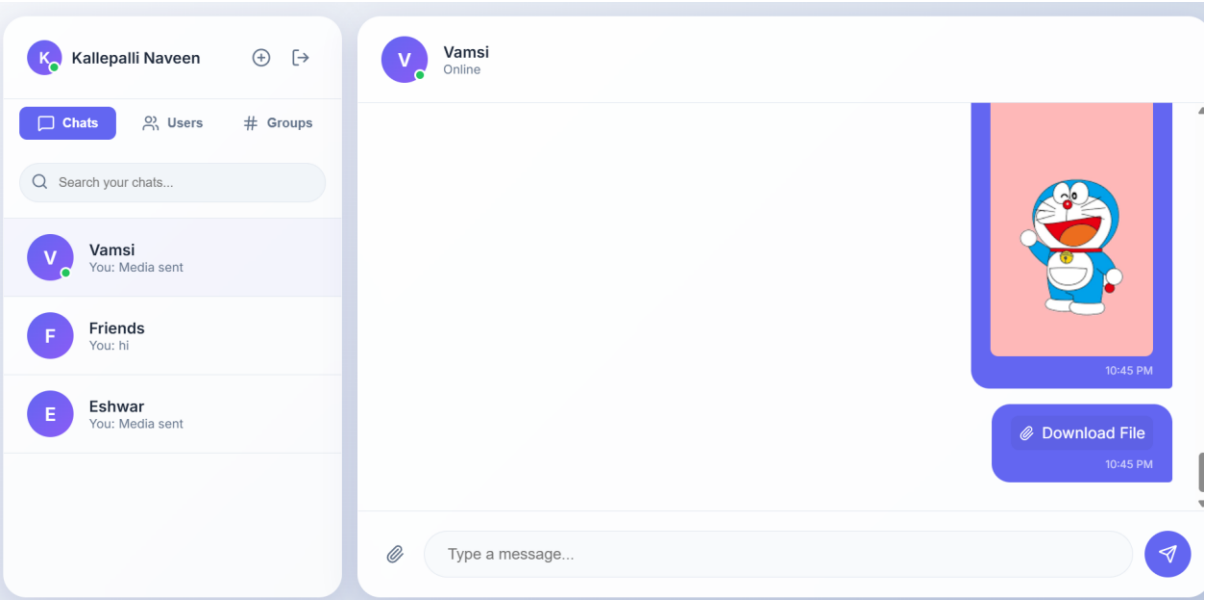


3.5 Media & File Sharing

Users can share more than just text. The platform supports:

* Images: Rendered directly within the chat bubble.

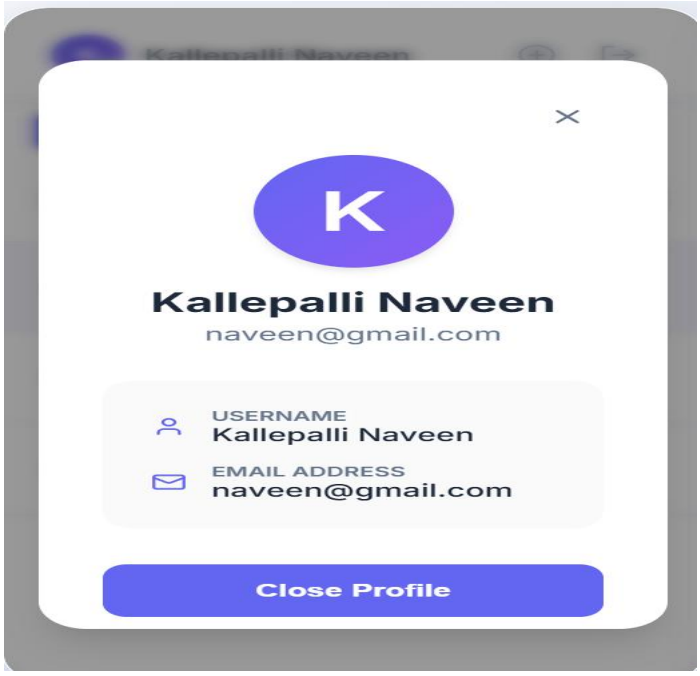
* Files: Documents (PDF, ZIP, etc.) appear as downloadable links with file icons.



3.6 User Profile Details

Users can click on their name/avatar in the sidebar to view their profile metadata, such as email and the date they

joined the platform.



---

4. Implementation Highlights

* Event-Driven Communication: Used Socket.io to handle room-based messaging, ensuring messages only reach the intended participants instantly.

* Data Persistence: MongoDB ensures that even if a user refreshes their page or logs out, their entire chat history is retrieved instantly upon return.

* Unified Search Logic: Implemented a robust filtering system that searches both local user chats and global database records simultaneously.

* Responsive UI: The UI uses a flexible layout that remains functional and aesthetically pleasing across various screen sizes.

5. Fulfillment of Requirements

The application successfully fulfills all requirements:

1. Real-time Chat: Messages deliver instantly via WebSockets.
2. Rooms/Private Chats: Supported through dynamic room creation.
3. Media Option: File/Image attachment feature is fully integrated.
4. Persistence: All data is saved in MongoDB.
5. Authentication: Secure JWT implementation.
6. Responsiveness: Mobile-friendly CSS architecture.

6. Conclusion

The application demonstrates a professional-grade implementation of the MERN stack. It handles high-concurrency real-time events and data persistence effectively, providing a scalable solution for modern digital communication.